Preliminary Task 1

Brooks Piper

2025-11-17

0. Get Dependencies

```r
require(tidyverse)
```

```
## Loading required package: tidyverse

## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.1     v stringr   1.5.2
## v ggplot2   4.0.0     v tibble    3.3.0
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.1.0
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
require(tidytext)
```

```
## Loading required package: tidytext
```

```r
require(textstem)
```

```
## Loading required package: textstem
## Loading required package: koRpus.lang.en
## Loading required package: koRpus
## Loading required package: sylly
## For information on available language packages for 'koRpus', run
##
##   available.koRpus.lang()
##
## and see ?install.koRpus.lang()
##
##
## Attaching package: 'koRpus'
##
## The following object is masked from 'package:readr':
##
##     tokenize
```

```r
require(rvest)
```

```
## Loading required package: rvest
##
```

```
## Attaching package: 'rvest'
##
## The following object is masked from 'package:readr':
##
##     guess_encoding
```

```r
require(qdapRegex)
```

```
## Loading required package: qdapRegex
##
## Attaching package: 'qdapRegex'
##
## The following object is masked from 'package:dplyr':
##
##     explain
##
## The following object is masked from 'package:ggplot2':
##
##     %+%
```

```r
require(stopwords)
```

```
## Loading required package: stopwords
```

```r
require(tokenizers)
```

```
## Loading required package: tokenizers
```

```r
require(tidymodels)
```

```
## Loading required package: tidymodels
## -- Attaching packages -------------------------------------- tidymodels 1.4.1 --
## v broom        1.0.10     v rsample      1.3.1
## v dials        1.4.2      v tailor       0.1.0
## v infer        1.0.9      v tune         2.0.0
## v modeldata    1.5.1      v workflows    1.3.0
## v parsnip      1.3.3      v workflowsets 1.1.1
## v recipes      1.3.1      v yardstick    1.3.2
## -- Conflicts ----------------------------------------- tidymodels_conflicts() --
## x qdapRegex::%+%()     masks ggplot2::%+%()
## x scales::discard()    masks purrr::discard()
## x qdapRegex::explain() masks dplyr::explain()
## x dplyr::filter()      masks stats::filter()
## x recipes::fixed()     masks stringr::fixed()
## x dplyr::lag()         masks stats::lag()
## x yardstick::spec()    masks readr::spec()
## x recipes::step()      masks stats::step()
```

```r
require(knitr)
```

```
## Loading required package: knitr
```

1. Get Preprocessed data

```r
# Load data
load('../data/claims-cleaned.RData')
load('../data/claims-cleaned-headers.RData')
```

2. Model with Logistic Principle Component Regression

```
# Setup model fitting function
lpcr_accuracy <- function(data_dtm){

  # Setup train/test split
  split <- initial_split(data_dtm, prop = 0.8)
  train <- training(split)
  test <- testing(split)

  # Separate covariates from response
  x_train <- train %>%
    select(-.id, -bclass) %>%
    select(where(is.numeric)) %>%
    as.matrix()

  y_train <- train$bclass

  x_test <- test %>%
    select(-.id, -bclass) %>%
    select(where(is.numeric)) %>%
    as.matrix()

  y_test <- test$bclass

  # Perform PCA
  col_sds <- apply(x_train, 2, sd)
  PCA_cols <- which(col_sds > 0 & !is.na(col_sds))

  x_train_PCA <- x_train[, PCA_cols, drop = FALSE]
  x_test_PCA <- x_test[, PCA_cols, drop = FALSE]

  pca <- prcomp(x_train_PCA, center = TRUE, scale. = TRUE)

  ## Add logic to extract EITHER 50 pcs or enough to capture 80% of variance
  var_exp <- pca$sdev^2
  prop_var <- var_exp / sum(var_exp)
  cum_var <- cumsum(prop_var)
  k80 <- which(cum_var >= 0.80)[1]
  k <- min(max(k80, 50), ncol(pca$x))
  train_pc <- pca$x[, 1:k, drop = FALSE]
  test_pc <- predict(pca, x_test_PCA)[, 1:k, drop=FALSE]

  # Perform logistic principle component regression
  train_df <- as_tibble(train_pc) %>%
    mutate(bclass = y_train)

  test_df <- as_tibble(test_pc) %>%
    mutate(bclass = y_test)

  lpcr_fit <- glm(formula = bclass ~ ., family = "binomial", data = train_df)
```

```r
  # Create prediction probs with 0.5 rule
  probs <- predict(lpcr_fit, test_df, type = "response")
  labels <- levels(y_train)
  preds <- ifelse(probs > 0.5, labels[2], labels[1]) %>%
    factor(levels = labels)

  # Calculate accuracy
  return(mean(preds == y_test))
}
```

```r
# Get models' accuracy
accuracy_paragraph_lpcr <- lpcr_accuracy(dtm_paragraph)
```

```
## Warning: glm.fit: algorithm did not converge
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```r
accuracy_header_lpcr <- lpcr_accuracy(dtm_header)
```

```
## Warning: glm.fit: algorithm did not converge
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```r
# Tabulate results
table <- data.frame(
  Strategy = c("Paragraph Only", "Paragraph and Header"),
  Accuracy = c(accuracy_paragraph_lpcr, accuracy_header_lpcr)
)

kable(table, main = "Binary Accuracy by Scraping Strategy")
```

Strategy	Accuracy
Paragraph Only	0.6987342
Paragraph and Header	0.6690476